Gaps and future work

`docs/TESTING.md` says what each test tier **proves**. This page says what **nothing here proves yet** — which parts of the LA9310 DFE the two test artefacts do not touch, what it would take to close each one, and the traps waiting in each.

It also records three things that must not be lost, because each of them is a place where the tests looked green while the thing under test was broken:

- the **64-sample block rule** — why `CP_LEN` is 64 and must stay a multiple of 64;
- the **retraction** of the old "implementation loss" figure;
- the **vacuous-test lesson** from the loopback verdict.

Read those three first. Everything else is engineering.

The two artefacts, and what they cover today

`firmware/src/main_test.c` — the on-chip loopback

Driven by `cosim/scripts/verify_dfe_loopback.py` (`--stages` compares every pipeline stage against the Python oracle). One tone at `sc[0]`, `0x2000` in SM16, through the real firmware path:

```
packed subcarriers -> in-place concat -> zero guard -> iFFT-1024 (sc128)
    -> bit-reversal (software)         [TX]
    -> FFT-1024 (sc128) -> bit-reversal (REAL IPPU microcode, bitRev1024sEbyE)
    -> subcarrier extraction            [RX]
```

Verdict: `dominant_sc == 0` **and** `dominant_e > 0` **and** `guard_max_e < GUARD_E_THRESH`. Measured: `dominant_e = 236196`, guard floor 16, threshold 256. All 8 stages bit-exact (0.00 LSB) against the oracle.

Covers: the subcarrier mapping, both FFT kernels, the RX IPPU bit-reversal, and the spectral cleanliness of the round trip. **Does not cover:** the cyclic prefix, windowing, rate conversion, QEC, phase compensation, CFO, any channel at all. It is one tone, no noise, no impairments.

`cosim/scripts/dfe_cosim.py` — the link-level BER co-simulation (tier 9)

```
TX (VSPA) -> +CP -> 2x interpolate (VSPA) -> AWGN -> 2x decimate (VSPA) -> -CP -> RX (VSPA)
```

Every DSP stage is the real kernel on the in-process ISS, linked from `firmware/lib/lib_DFE_Ref_vspa2.elb` — the same library the firmware links. Only the channel is Python. `cosim/scripts/kernel_check.py` gates each stage against an independent Python model (`cosim/vspa_model/`) before any BER number is believed.

The geometry, and it is load-bearing (`cosim/scripts/dfe_cosim.py:44-63`):

symbol	value	why
`N`	1024	`POINTS_FFT` for the active build
`NSC`	61	the runtime subcarrier count the whole repo simulates
`CP_LEN`	**64**	see the 64-sample block rule below
`N_CP`	1088 = **17 x 64**	symbol + CP, into the interpolator
`N_UP`	2176 = **34 x 64**	at the 2x rate
`DELAY`	9	cascade group delay, compensated by a cyclic roll
`AMPLITUDE`	8192	QPSK radius in SM16 counts (1/4 full scale)

Covers: the TX chain, both rate converters, an AWGN channel, the RX chain, and whether the link carries bits. **Does not cover:** anything analogue (I/Q imbalance, CFO), any timing error, any channel that is not flat AWGN, any slot structure — and, at its default depth, any BER below about 1e-3.

> The channel impairments and the QEC stage are being wired in from the older monorepo as this is written. `cosim/kernels/compute_qec_td.c` and `cosim/vspa_model/qec.py` exist and are gated by `kernel_check.py`; whether the corresponding CLI flags are in the sweep yet, `dfe_cosim.py --help` is the authority. Where a gap below says "wire it in", that is the remaining step.

Three things that must not be lost

1. The 64-sample block rule

The SX rate-conversion kernels process whole 64-sample input blocks. A length that is not a multiple of 64 comes back with a silently ZEROED TAIL.

`X2_interp_tap32_filter.sx:70` and `ddc2x4x.sx:35` (both under `submodules/la931x_vspa_common/vspa-lib/resample_ddc_interp/src/`) compute their loop count as

```
sr g0, g0, 6          ; iterations = floor(n_samples / 64)
```

A partial trailing block is **dropped**, not processed. And because the host zero-fills the output buffer before the call, the dropped samples read back as a clean **exactly 0.0** — not as garbage. No fault. No warning. The chain looks like it lost a little energy, not like it broke.

Why 64, and why it is not a tunable. A VSPA vector line is 128 bytes = 32 complex `fx16` samples; the kernels' inner loop consumes two lines per iteration. 64 is the AU vector width times the software pipeline depth — it is a property of the assembly, not a constant anyone chose. `firmware/lib/filter.h:19` states it plainly: "*number of input samples must be multiple of 64 samples*".

What it cost. The original co-simulation used `CP_LEN = 20`, so it fed the interpolator `1024 + 20 = 1044 = 16*64 + 20` samples. The last 20 input samples were never filtered:

output range	max error vs the Python model
0 .. 2047	**1.0 LSB**
2048 .. 2087	**19055 LSB** — exactly zeros

About **2 % of every symbol's time domain was zeroed on both sides, before the channel even started**, and the BER curve still looked plausible.

The rule. `N` is already a multiple of 64, so `CP_LEN` must be one too. `CP_LEN = 64` is the smallest choice that is both a multiple of 64 and larger than the cascade group delay (9) with room left for a timing-offset margin. Any change to the geometry must satisfy

```
assert (N + CP_LEN) % 64 == 0
assert (2 * (N + CP_LEN)) % 64 == 0
```

and those asserts belong **in code**, not in a comment. This is the single constraint that a future editor is most likely to "optimise" away — a 20- or 72-sample CP looks more standards-like, and it will quietly re-break the chain.

2. The retraction

The old "implementation loss grows from +0.4 dB to +2.8 dB with SNR" result is WITHDRAWN. Do not reproduce it. Do not quote it.

The old monorepo's deep run (1000 symbols x 8 SNR points) reported an implementation loss that grew monotonically with Eb/N0 and concluded this was "*the signature of a fixed-point noise contribution that is constant in absolute terms and therefore dominates more and more at high SNR*".

That run was measured at `CP_LEN = 20` -> `N_CP = 1044`. Per the block rule above, **the last 40 output samples of every interpolated symbol and the last 20 of every decimated one were exactly zero**. That error is deterministic and **SNR-independent** — which is *precisely* the fingerprint that was read as quantisation noise. The experiment cannot distinguish SM16 quantisation from a zeroed tail. The conclusion does not survive the bug; neither do the numbers.

Consequently: this repo has no implementation-loss figure, and must not state one until a deep run is made on the corrected geometry.

Acceptance criteria for that re-run:

- `cosim/scripts/kernel_check.py` green for **every** stage first — a BER curve built on unverified stages is a drawing, not a measurement;
- > = 1000 symbols/point over >= 8 points;
- **errors and bits recorded per point**, not just the ratio;
- any point with fewer than ~50 errors reported as an **upper bound**, not a BER (`dfe_cosim.py` already draws these as downward arrows rather than inventing a point — keep that);
- the question the old run appeared to answer and did not: **does the curve flatten into an error floor below ~1e-5?** That is still open.

Cost, measured on the corrected chain in this repo: TX + interpolate **~2.5 s/symbol** (hoisted out of the SNR loop, so once per symbol, not once per (symbol, point)); decimate + RX **~2.1 s per (symbol, SNR point)**. 1000 sym x 8 points projects to **~5.4 h sequential, ~40 min across 8 processes**.

3. The vacuous-test lesson

The loopback's verdict used to pass unconditionally, for any pipeline whatsoever. The old check read the recovered subcarriers as

```
int re = rx_sc_out[i].real;      /* .real is a __fx16 */
```

That is a **numeric conversion**, not a bit copy: a `__fx16` is a fraction in [-1, 1), so every `|value| < 1` truncates to **0**. The energy `e = re*re + im*im` was therefore always 0, `max_e` stayed 0, `dominant_sc` never moved off its initial value of 0 — and `test_result = (dominant_sc == 0)` reported **PASS**. The test asserted nothing. It would have passed on an all-zero pipeline, on garbage, on a build with the FFT removed.

Fixed in `firmware/src/main_test.c`, three ways at once:

1. the SM16 magnitudes are read through a **halfword view** (`bits & 0x7FFF`) — the FFT kernels emit sign-magnitude, not two's complement;
2. `dominant_sc` starts at **-1**, and PASS additionally requires **`dominant_e > 0`** — an all-zero pipeline now FAILs loudly;
3. a **guard-band energy assertion** (`guard_max_e < 256` , against a measured floor of 16 and a tone energy of 236196) — because "the peak is in the right bin" says nothing about where the rest of the energy went.

Both negative controls behave: threshold 8 -> FAIL on the noise floor; tone moved to `sc[5]` -> `dominant_sc = 5` , FAIL.

The generalisation, and it is the reason this section exists: on VSPA, a *type* conversion where you meant a *bit* reinterpretation produces zeros, and zeros produce a clean, plausible, passing test. The same trap sits in every host poke into DMEM (`.bss` is zeroed by crt0 — see the constraints section) and in every `__fx16` assignment. When a test is suspiciously green, first ask whether it can fail at all.

The gap matrix

Three states, and no fourth:

- **exercised** — the real thing runs;
- **stand-in** — something runs, and the stand-in is *named*;
- **not exercised** — nothing runs.

There is deliberately no "closed with a caveat" state. That notation is where honest caveats go to die.

#	LA9310 feature	firmware symbol	loopback	co-simulation	state	priority
1	CP insertion (TX)	IPPU, fused with bitrev	no	Python `add_cp`, fixed 64	stand-in — fixed CP, not slot-aware LCP/SCP	**high**
2	CP removal (RX)	`read_offset_fft_in`	no	Python `strip_cp` + delay roll	stand-in — same	**high**
3	TX windowing	`txWindowing_circ_w16`	no	scalar C re-implementation in `compute_dfe_tx.c`	stand-in — and applied in the **wrong place** (see gap 3)	medium
4	IPPU bit-reversal	`bitRev1024sEbyE`	**RX: real IPPU**; TX: software	software both ways	stand-in (TX blocked on kernel availability)	medium
5	2x TX interpolation	`X2_interp_tap32_filter`	no	**exercised** (real lib kernel)	exercised	—
6	2x RX decimation	`decimator_2x_8_Taps_asm`	no	**exercised** (real lib kernel, 8-tap)	exercised; **64-tap variant not**	low
7	TX QEC (pre-distortion)	`qec_opt_asm`	no	no	**not exercised**	medium
8	RX QEC	`qec_opt_asm`	no	**exercised** — real `qec_opt.sx` on the ISS, `--qec on`	exercised; coefficients are a runtime port, not baked in	—
9	Phase compensation	`phcom_asm`	no	Python post-FFT ramp, `--phase-comp on`	stand-in — the real `phcom_asm` still unused (see gap 9)	medium
10	CFO estimation	`fineCFO_rc`	no	Python CP-autocorrelation, `--cfo` / `--cfo-correct`	stand-in — and an honest one: it is variance-dominated below ~10 dB (see gap 10)	medium
11	Cell tracking (PSS/SSS)	`sssdet_targeted_hpgemv_rc_v1`	no	no	**not exercised** — blocker	low

#	LA9310 feature	firmware symbol	loopback	co-simulation	state	priority
					is stimulus, not kernels	
12	Channel estimation + equalization	`chEst`	no	no	**not exercised** — needs pilots + a multipath channel	low
13	Timing offset / CP guard	`read_offset_fft_in`	no	**exercised** — `--timing-offset K`, undone by the gap-9 ramp	exercised (in-guard regime only)	low
14	Multi-symbol statistical depth	—	1 tone	mechanism **done** (checkpoint / resume / shard); the corrected deep run is the open item	see the retraction — the number, not the machinery, is what is missing	**high**
15	Guard-band energy check	—	**exercised** (+ 2 negative controls)	implicit	exercised	—
16	Host <-> VSPA messaging	`host_vspa_msg.c`	no	no	out of scope (HW)	—
17	AXIQ / antenna buffers	`Ant_TxRx_Buffer_manager`	no	no	out of scope (HW); tier 8 anchors the DMA/SM16 semantics against silicon	—

Which kernels are actually available. Verified by symbol dump of `firmware/lib/lib_DFE_Ref_vspa2.elb`:

- **present:** `_qec_opt_asm`, `_phcom_asm`, `_fineCFO_rc`, `_chEst`, `_sssdet_targeted_hpgemv_rc_v1`, `_txWindowing_circ_w16(_vecaligned)`, `_X2_interp_tap32_filter`, `_decimator_2x_8_Taps_asm`, `_single_tone_asm`;
- **absent:** `X2_decim_tap64_filter` (the header `firmware/lib/X2_decim_tap64_filter.h` exists, the symbol does not), and any `zf_equalizer`.

So gaps 8, 9, 10, 11 and 12 are **not** blocked on kernels — a `compute_*.c` harness is ~30 lines. What blocks them is stimulus (11, 12) or chain structure (9, 10). Gap 6's 64-tap variant is the one that genuinely needs an `.sx` from `submodules/` on the link line.

Per-gap detail

Gaps 1+2 — cyclic prefix, and the slot-aware LCP/SCP schedule

What it is. OFDM prepends a copy of each symbol's last CP samples. On LA9310 the length depends on the symbol's position in the CP period: for the active build (`BANDWIDTH_25M_30K_61_61` -> `SCS_30K` / `BANDWIDTH_25M` , `firmware/include/pair_config.h`) `NUM_SAMPLES_LCP = 352/4 = 88` for symbol 0, `NUM_SAMPLES_SCP = 288/4 = 72` for symbols 1..13, with `NUM_SYM_PER_CP_PERIOD = 14` .

(Note: the header comment in `firmware/src/main_test.c` says LCP = 80. That is the `SCS_15K` / `BANDWIDTH_10M` block's value; the active config gives 88. The comment is stale — the code does not depend on it.)

What the tests do. The loopback skips the CP entirely. The co-simulation uses a **fixed 64-sample CP** in Python, which is not any of the real lengths.

What closing it takes — and the trap. `1024 + 88 = 1112` and `1024 + 72 = 1096` are **neither of them multiples of 64**. Driving the rate converters one slot-aware symbol at a time therefore **re-creates the exact bug that invalidated the old BER run**. But a whole CP period *is* exact:

```
14*1024 + 88 + 13*72 = 15360 = 240 * 64        (and 30720 = 480 * 64 at the 2x rate)
```

So the closure is: **drive the interpolator and decimator a CP-period at a time, carrying the filter `history` / `filtState` buffers across calls** instead of zeroing them per call (`cosim/kernels/compute_x2_interp.c` , `compute_decim_2x_8tap.c` both cold-start today). That is what the real DFE does with its circular buffers, and it is the single highest-value structural change in this document, because it also:

- unblocks **gap 9** (`phcom_asm` needs the symbol index within the CP period);
- makes the **CFO model continuous** rather than per-symbol (gap 10);
- removes the per-symbol filter cold-start transient — today the first ~15 samples of every symbol see zero history. That is currently harmless *only* because the transient is shorter than the 64-sample CP that gets discarded. It stops being harmless the moment the geometry changes.

The IPPU `cp_insert` / `cp_remove` path (the real DFE fuses CP insertion with the TX bit-reversal) remains a further refinement; the Python copy is faithful in value, just not in engine.

Gap 3 — TX windowing, and where it is applied

What the firmware does (`firmware/src/DFE_func_tx.c:397`):

```
txWindowing_circ_w16(dst, tx_windowing_history_buff, real_window,
                     ifft_cp_size/4, iFFTout_CFRinout, IFFT_CIRC_SIZE);
```

— a raised-cosine window applied **over the CP region of the circular TX buffer**, with a **history buffer** carrying the taper across symbol boundaries. It shapes the *transition* between symbols; the N payload samples the receiver FFTs are untouched.

What the co-simulation does (`cosim/kernels/compute_dfe_tx.c:65`): a scalar C re-implementation applies the fade-in to the **first 16 samples of the symbol body** and the fade-out to its **last 16** — *before* the CP is prepended. So the taper lands **inside the payload the receiver demodulates**, and the CP (a copy of the tail) carries the fade-out.

Two consequences, and the second one matters:

1. The lib's own `txWindowing_w16_vecaligned` is present and unused. Swapping the scalar loop for it is a ~10-line change and makes the stage real.
2. The current placement is a **distortion of the payload**, not a model of the firmware. It is a candidate contributor to any implementation loss — which is a second, independent reason not to quote an implementation-loss figure until it is fixed. Closing it properly needs the window applied across the CP boundary with the history carried over, i.e. it wants gap 1's CP-period drive.

Spectral-mask measurement (adjacent-channel power) is not asserted anywhere. The window's effect on BER in flat AWGN is nil by design; its effect on out-of-band emission is the thing it exists for, and nothing measures it.

Gap 4 — IPPU bit-reversal

The **loopback's RX bit-reversal runs real NXP IPPU microcode** (`bitRev1024sEbyE` , from `submodules/.../vspa-lib/bitRev/asm/`), not a software proxy. Everything else — the loopback's TX, and both directions in the co-simulation kernels — uses `sw_bitrev` .

TX cannot use `bitRev1024sEbyE` , and this is not laziness. That procedure is bit-reversal **plus an N/2 shift** (the "s" in its name). On RX that shift is exactly right: it DC-centres the spectrum, so the active subcarriers become one contiguous run at `N/2 - NSC/2` and extraction collapses from two wrap-around copies into one (confirmed: the `sc[0]` tone lands at bin 482 = 512 - 61/2). On TX the same shift would emit a time-domain symbol **cyclically rotated by N/2** — a different signal, whose spectrum picks up a `(-1)^k` factor that flips the sign of every odd subcarrier.

> **The trap, and it is live.** With the single tone at `sc[0]` (even bin 994) the TX stage is **invariant** under a 512-sample rotation. Use the sEbyE kernel for TX and the loopback still compares **bit-exact — while being wrong**. Verified. Do not "simplify" here.

To close the TX half: vspa-lib ships no pure 1024-point bitrev IPPU procedure, and the app's own `bitRev1024_reverseIn_linearOut_vspa2_b*` is fused with CP insertion and circular-DMA addressing (it stalls at IPPU `exec_pc` 14 on the ISS when driven without the TX DMA context). So this is **blocked on NXP kernel availability**, not on the simulator: it needs either a pure-bitrev IPPU procedure or the full TX DMA context. Ask NXP for the former.

Gap 6 — the 64-tap decimator

`NUM_2XDOWN_TAPS = 8` in `firmware/include/proj_config.h` ; 64 is selectable and inactive. The 8-tap/64-tap trade-off (stopband attenuation vs core load) is untested.

Unlike the other open kernels, this one cannot be closed against the library: `X2_decim_tap64_filter` is **not an exported symbol** of `lib_DFE_Ref_vspa2.elb` (only `decimator_2x_8_Taps_asm` and `decimator_4x_8_Taps_asm` are). Closing it needs the `.sx` from `submodules/` on the link line — the same pattern `cosim/kernels/Makefile` already uses for QEC. Do not promise a 30-line harness here.

Gaps 7+8 — QEC (transmit and receive)

Both are enabled by default on silicon (`DEFAULT_TX_QEC_DISABLE = 0` , `DEFAULT_RX_QEC_DISABLE = 0`).

The RX half is ready to wire in. `cosim/kernels/compute_qec_td.c` links the real submodule assembly (`submodules/la931x_vspa_common/vspa-lib/qec/src/qec_opt.sx` , with `-DVSPA2` selecting the "2nd round optimization" body), and `cosim/vspa_model/qec.py` is its independent model. `cosim/scripts/kernel_check.py` gates it two ways: **bit-exact vs the model**, and **does it actually undo an injected imbalance** (23.1 dB -

> 53.8 dB). The second assertion is the one that matters — a kernel that silently ignored its parameter line would pass the first.

Wiring: the QEC sits at the **1x rate, after CP removal, before the RX FFT**.

```
Port("input_td",   "input_td",   1024)                 # sm16_complex
Port("qec_params", "qec_params", 32, enc="u32")        # 32 float32 bit patterns
-> Port("output_td", "output_td", 1024)
```

Correcting at 1x for an impairment injected at 2x is **exact, not approximate**: the imbalance is memoryless and real-linear, the decimator's taps are real, so the two operations commute.

Three things to know about the parameter line (`qec_params_opt_t`, one 128-byte vector line = 32 float32):

- layout `0=f1 1=f4 2=pad0 3=f2 4=dcoff_I 5=dcoff_Q`, `6..11` a backup copy, `12..31` zero. **`pad0` (word 2) and `pad1` (word 8) must stay 0** or the kernel becomes stateful;
- transfer function with `pad0 = 0`: `out_I = f1*I + dcoff_I`, `out_Q = f2*I + f4*Q + dcoff_Q`. For an injected imbalance `(eps, phi)`: `f1 = 1`, `f4 = 1/((1+eps)*cos(phi))`, `f2 = -tan(phi)`;
- **the scaling trap:** the kernel's samples are half-fixed (full scale 1.0) while the co-simulation carries SM16 **counts** (full scale 32768). `f1` / `f4` / `f2` are ratios and are the same number in both domains — **`dcoff` is not**. A DC of `d` counts must be sent as `d/32768.0f`. (Confirmed on the ISS: `dcoff_I = 655/32768` lands as exactly +655 counts.)

`num_samples` is rounded **up** to a multiple of 32 internally (`qec_opt.sx`: `add g0,31; sr g0,g0,5`) and the kernel then writes that many lines — so a non-multiple **writes past the end of the output buffer**. 1024 is safe; 1088 would be too. It is the mirror image of the 64-block rule: the interpolator under-processes, QEC over-writes.

The TX half (gap 7) is not exercised at all. The same ELF serves the TX position — it is the same `qec_opt_asm`. What is missing is a **TX-side imbalance to pre-distort against**: a second imbalance instance in the channel model, applied *after* the TX QEC. Until that exists, exactly one of the two QEC instances the silicon runs is being tested, and the row must not be marked closed.

Gap 9 — phase compensation, and a correction to how it was specified

`phcom_asm` is not what the old gap document said it was.

`phcom_asm(out, in, coeff, nL)` applies **one complex scalar per symbol**, indexed by the symbol's position in the 14-symbol CP period:

- `firmware/include/global_var.h:36` — `float phcom_coeff_tx[NUM_SYM_PER_CP_PERIOD][2]`
- `firmware/src/DFE_func_tx.c:546` — TX, on the subcarrier buffer
- `firmware/src/DFE_func_rx.c:888` — RX, on the time-domain symbol

That is **NR carrier phase compensation** — a common-phase-error rotator. It is **not** a per-subcarrier linear ramp, and it cannot implement one.

The per-subcarrier ramp `exp(+j*2*pi*m*k/N)` (with `m = j - NSC//2`) is the correction for a **timing offset** (gap 13), which is a different physical quantity. Keep it in float, in Python. Do not route it through `phcom_asm`, and do not go looking for `zf_equalizer_1tap` — it is not in the library either.

To close gap 9 with the real kernel: `_phcom_asm` *is* in the library and the harness is ~30 lines. What blocks it is that the co-simulation has **no symbol index within a CP period** — every symbol is an independent buffer. So gap 9 depends on gap 1. (The old blocker — "the native backend only supports single-input kernels" — is **gone**: `cosim/dfe/kernel_runner.py` takes a *list* of input `Port`s.)

Gap 10 — CFO, and why the obvious estimator measures nothing

Injection is trivial and ports as-is: a CFO of `frac` subcarrier spacings is `exp(+j*2*pi*frac*n/N)`, `N = 1024`. It belongs on the **decimator's CP-present output, before `strip_cp()`** — that is where a receiver LO offset physically sits. Assert `|frac| < 0.5` (the estimator's `arg()` wraps beyond that; the old CLI never checked).

The estimator is where the old repo went wrong, and it is worth understanding before anyone re-derives it. The van-de-Beek / Moose CP autocorrelation is sound: the CP is a copy of the payload tail `N` samples away, so `arg(sum conj(cp[n]) * tail[n]) = 2*pi*frac`. But the old caller **reconstructed the CP by copying the payload's own tail**, and only *then* injected the ramp. The two correlation windows were therefore **the same samples with the same noise**, differing only by the ramp just added. Measured on this repo's chain: it returns the injected value **exactly, standard deviation 0.0000, at every SNR**. It is not an estimator; it is an echo. And it fails **silently**: inject the CFO in the physically correct place and the same code returns `+0.0000` for a `+0.05` offset — the corrector then does nothing, the BER quietly degrades, and nothing reports an error.

The honest estimator exists only at this repo's geometry. Correlate the **raw 1088-sample decimator output**, whose CP and payload tail carry *independent* noise, offsetting for the group delay:

```
def estimate_cfo_cp_raw(rx_cp, start=DELAY + 2):
    L = len(rx_cp) - start - N        # 1088 - 11 - 1024 = 53 usable samples
    c = np.sum(np.conj(rx_cp[start:start + L]) * rx_cp[start + N:start + N + L])
    return float(np.angle(c) / (2 * np.pi))
```

`L = N_CP - N - DELAY = 55`, minus the 2 filter-transient samples at the head (measured: 92 % and 30 % mismatch at offsets 9 and 10, under 3 % from 11 on) = 53. Measured over 40 trials with `frac = +0.0500` injected:

Eb/N0	mean estimate	std
12 dB	+0.0523	0.0218
6 dB	+0.0548	0.0667
0 dB	+0.0242	0.1828

That is what a CP estimator is supposed to look like. **At the old broken geometry this is impossible:** `L = 1044 - 1024 - 9 = 11`, 9 usable. Nine samples is not an estimator — which is part of why the old repo reconstructed the CP in the first place. **The 64-sample CP does not merely fix a bug; it enables a feature.**

The real kernel stays out of reach. `_fineCFO_rc` is in the library, but it estimates CFO from PSS/SSS **frequency-domain** channel estimates (256 complex bins each) — it needs an NR sync burst this flat-AWGN co-simulation does not carry (gap 11). The CP estimator is therefore a **labelled Python stand-in for a kernel this repo owns but cannot feed**, and it must be labelled that way wherever its results appear.

Gap 13 — timing offset

For `|K| <= CP_LEN` the cyclic prefix guards the symbol, so shifting the FFT window is an **ISI-free cyclic shift** — `np.roll(rx_td, K)` on the 1024-sample payload, after `strip_cp()` and before the QEC/FFT, in both the VSPA and model paths. The FFT turns it into a per-subcarrier phase ramp, which the gap-9 ramp inverts (measured on the model: `K = +20` -> EVM 1.387 without, 0.0062 with).

The safe range is `|K| <= CP_LEN - DELAY ~= 55`. Note this is a *model* of a window shift, not a true re-slice of the 1088-sample buffer — a true re-slice for `K > 0` would need samples past index 1087, which the group

delay has already truncated. For `|K| <= CP` the cyclic roll is exact, so it is the right call; past that it is not, and the assertion should say so.

Keep the sign convention of the roll glued to the sign convention of the compensation ramp. They are inverses of each other only if both use the same one.

Gap 14 — statistical depth, checkpointing and sharding

The default is 10 symbols/point = 1220 bits. Anything below ~1e-3 is noise. See **the retraction** above for what depth is required and what it costs.

An ~40-minute sharded run should not be all-or-nothing, and the old repo's checkpoint machinery is the one piece of it that is pure infrastructure and worth carrying: chunked sweep, atomic checkpoint flush (`.tmp` + `fsync` + `os.replace`), a config fingerprint that **refuses** a `--resume` whose configuration differs, and a `--merge` that refuses to combine shards whose configs disagree or whose points overlap (double-counting a point halves its BER with no other symptom).

But the RNG keying does not transplant unchanged, and getting it wrong is silent. The invariant that makes a resumed or sharded run element-wise identical to an uninterrupted one is: *every draw must be a pure function of* `(seed, point, chunk, position-in-chunk)` *and of nothing else* — in particular not of how many draws came before it. This repo's sweep **hoists TX + interpolate out of the SNR loop** (`cosim/scripts/dfe_cosim.py:181-199`) and reuses one symbol list across all SNR points, which is a real ~4x saving. So a symbol's **bits are shared by every point and cannot be keyed by point**. The stream must be **split in two**:

```
bits  <- SeedSequence([seed, BITS_DOMAIN, chunk])           # point-INdependent
noise <- SeedSequence([seed, NOISE_DOMAIN, point, chunk])   # point-keyed
```

With the split, sequential == sharded-by-point == sharded-by-chunk == resumed, element-wise. Without it — a single streaming `rng` , as the sweep originally had — recomputing point 2 alone gives different noise than it had in the full sweep, and every shard is silently a different experiment.

STATUS: done. `rng_bits()` / `rng_noise()` implement exactly this split, and it is verified element-wise, not merely asserted: monolithic, SIGTERM-interrupted + `--resume` , and 2-shard + `--merge` all return 48/488, 15/488, 2/488 on the same three points.

Two more, both cheap and both load-bearing:

- the fingerprint **must include the geometry** (`N` , `NSC` , `CP_LEN` , `DELAY` , chunk size). It is precisely what stops someone resuming a `CP_LEN = 64` run into a checkpoint written under the broken 1044 geometry and splicing two different experiments together. The shard *selection* must **not** be in the fingerprint, or `--merge` can never prove two shards belong together;
- the sweep used to write `docs/figures/ber_vs_snr.png` unconditionally at the end of `main()` — a TRACKED deliverable that `README.md` and `docs/TESTING.md` embed next to a table quoting its depth. Fan out 8 shards and they race on that one file; run a 2-symbol smoke test and you silently degrade it. **STATUS: done.** A shard now reports its points and returns (no summary, no plot, no BER floor over a partial grid), and `run_deep_ber.sh` passes `--figure` so a deep run writes next to its checkpoints instead of into `docs/` .

Sharding must be by **process**: `cosim/dfe/kernel_runner.py` holds a single process-global `NativeModel` , because the model DLL keeps per-device state. Sharding by **chunk** rather than by SNR point preserves the TX-once saving (a point-shard must redo the shared TX work for every symbol it needs); both are element-wise sound under the split keying.

Gaps 11+12 — PSS/SSS cell tracking, channel estimation, equalization

`CELLTRACK_ENABLE = 1` on silicon, and `_chEst`, `_sssdet_targeted_hpgemv_rc_v1` and `_fineCFO_rc` are **all in the library**. **The blocker is stimulus, not kernels**: cell tracking needs a multi-symbol sequence carrying an NR synchronisation burst, and equalization needs a pilot pattern plus a multipath channel — this co-simulation has a flat AWGN channel, in which equalization is a no-op.

The dependency chain is worth stating once: **gap 11 gates gap 10's real kernel** (`fineCFO_rc` consumes PSS/SSS channel estimates) **and gap 12** (which needs the acquisition to have happened). Closing 11 means writing an NR sync-burst generator for `cosim/vspa_model/`. It is the largest single item in this document and the one with the most kernels waiting behind it.

Gaps 16+17 — host messaging and AXIQ

Out of scope for software co-simulation: neither artefact can emulate the AXI mailbox protocol or the AXIQ FIFO hardware. The closest thing to coverage they will get is tier 8 (`cosim/tools/dfe_md5_anchor/`), which reproduces MD5s of dumps captured from **real LA9310 silicon** byte-exactly, and in doing so validates the DMA and SM16 semantics against the chip.

Two more items of hygiene

Restore the idiomatic `memcpy_dst_circ` in-place concat. The loopback's subcarrier packing (`firmware/src/main_test.c`, STEP 1) currently uses plain scalar loops. The real app uses the library's circular copy (`memcpy_dst_circ`, `firmware/src/DFE_func_tx.c:554`). Two defects used to hide behind that call — an unaligned `work_buf` (the lib's circular **vector** writes then land at wrong offsets) and `NSC = MAX_NUM_SUBCARRIERS = 780 > 2N/3` (the in-place concat self-clobbers). **Both are now fixed**: `work_buf` is `aligned(64)`, and the `NSC <= 2N/3` limit is a compile-time `#error`. So the library call should work again, and restoring it would make the loopback exercise the code path the firmware actually runs. The scalar version is what has been verified bit-exact stage by stage, so the swap must be re-verified with `verify_dfe_loopback.py --stages`, not assumed.

TX-side IPPU bit-reversal — see gap 4. Blocked on NXP kernel availability (`bitRev1024sEbyE` applies an N/2 shift: right for RX, wrong for TX; and the tone test cannot see the difference).

Roadmap, ordered by value

	Work	Why it is here	Cost
~~1~~	~~Wire the impairments + RX QEC into `dfe_cosim.py` ~~ **DONE** — `--iq-imbalance`, `--qec`, `--cfo` / `--cfo-correct`, `--timing-offset`, `--phase-comp`, each mirrored into the Python-model path	QEC and phase-comp measurably return the chain to baseline BER; the CFO estimator is kept but reported honestly (gap 10)	—
2	~~Split the RNG streams, add checkpoint/resume/shard~~ **DONE** (verified element-wise) — what remains is **the corrected deep BER run** itself	It is the only way to replace the retracted number with a real one	~2 h of compute, once
3	Slot-aware LCP/SCP, driven a **CP period at a time** with the filter history carried across calls	Unblocks gaps 9 and 13, makes CFO continuous, removes the cold-start transient — and is the one gap whose naive closure re-creates the 64-block bug	medium
4	Real `txWindowing_w16_vecaligned`, applied at the CP boundary (gap 3)	Today's placement distorts the payload the receiver demodulates. It is a second reason the implementation-loss figure is not yet quotable	small, after 3
5	Real `phcom_asm` (gap 9)	The kernel is in the library; it needs gap 3's symbol index	small, after 3
6	TX QEC pre-distortion (gap 7)	Exactly one of the two QEC instances the silicon runs is currently exercised	small
7	64-tap decimator (gap 6)	Needs the `.sx` on the link line — the library cannot supply it	small
8	Restore `memcpy_dst_circ` in the loopback	Hygiene: exercise the code path the firmware runs	small
9	PSS/SSS burst generator -> cell tracking, `fineCFO_rc`, `chEst`, equalization (gaps 10-12)	The largest item, and five library kernels are waiting behind it	large
—	TX IPPU bit-reversal (gap 4)	Blocked on NXP: no pure 1024-point bitrev IPPU procedure exists	blocked
—	Host messaging, AXIQ (gaps 16-17)	Hardware only	out of scope

Constraints that shape all of this

Every item above collides with at least one of these. They are collected here so that a future editor meets them before, rather than after.

1. **The 64-sample block rule.** `(N + CP_LEN) % 64 == 0`, or the rate converters silently zero the tail. See above. Assert it in code.
2. **Inputs are written at a breakpoint on `main()`.** `NativeKernelRunner` (`cosim/dfe/kernel_runner.py`) loads the ELF fresh per call and pokes DMEM *after* crt0 has zeroed `.bss`. A harness that initialises its

own input buffer in `main()` **overwrites what the host wrote** and computes on the compiled-in value — returning a clean, plausible, wrong answer with no error and no warning. Grep any new `compute_*.c` for assignments to a host-written buffer.

3. **SM16 is not two's complement.** VSPA assembly kernels use sign-magnitude (sign in bit 15, magnitude in 14:0). Read magnitudes as `bits & 0x7FFF` through a halfword view; a `__fx16` assignment is a *numeric* conversion (see the vacuous-test lesson).
4. **`NSC <= 2N/3`.** The in-place subcarrier concat self-clobbers past that (682 for N = 1024). Enforced by `#error` in `main_test.c`; `DfeSession` refuses `nsc > 682`.
5. **`qec_opt_asm`'s `num_samples` must be a multiple of 32** — it rounds *up* and writes that many lines, past the end of a shorter buffer.
6. **The interpolator's taps are in reverse-PAIR order** in the hardware buffer (`h[M-2],h[M-1], h[M-4],h[M-3], ...`). Not a no-op, palindrome or not — see `_unreverse_pairs` in `cosim/scripts/dfe_cosim.py`.
7. **IPPU:** microcode goes to memory space 2, its data to space 3; do **not** set `go[1]` (it is stepped from the VCPU's `sim_cycle`); its argument base is relative to the *model's* `ippu_dmem_base()`, not the app LCF's silicon split.
8. **One process-global `NativeModel`** — the model DLL keeps per-device state, so parallelism means **processes**, not threads.

Non-goals — do not re-add these

Each of these exists in the older monorepo for a reason that no longer holds here. Re-adding any of them would be a regression against a repo that is already more correct.

- **`params.hex` / baked-in kernel parameters / `rebuild=True`.** The QEC coefficients are a **runtime port** (`Port("qec_params", ..., enc="u32")`). One ELF, built once, serves every `(f1, f4, f2)`. There is nothing to rebuild and nothing to snapshot.
- **Resident worker subprocesses / `--backend` selection.** The ISS runs **in-process** via ctypes. There is one backend, no process group, no inherited pipes, and no `.close()` to call. (A SIGINT handler that sets a flag and returns is sufficient by construction: Python cannot interrupt the in-flight C call into the simulator.)
- **File I/O inside kernels.** The native ISS services only `write`/`clock`/`time`; `open` and `read` return -1. There is no `NATIVE_IO` handshake and no `fopen`/`fread` fallback — the host pokes DMEM.
- **`CP_LEN = 20` / `N_CP = 1044`.** See the 64-sample block rule. Any snippet copied from the old co-simulation's per-symbol loop carries this with it.
- **The old BER figures, result files and checkpoints.** See the retraction. They were measured on a chain with ~2 % of every symbol zeroed on both sides.